ЛР4. SCA: зависимости, SBOM и контейнер с Trivy

Время: 2 академических часа. **Что получится:** вы просканируете библиотеки, Dockerfile и образ, сформируете SBOM, отделите опасные находки от неприменимых, примените готовое исправление и настроите порог, по которому сканер останавливает конвейер.

Перед началом: выполнена ЛР3, запущен Docker.

Коротко о главном

- Большая часть приложения — **чужой код**: библиотеки и базовый образ. Их уязвимости — ваши уязвимости.
- **SCA** сверяет названия и версии компонентов с базами известных уязвимостей. Он не ищет новые ошибки, он отвечает на вопрос «есть ли у нас версии, про которые уже известно, что они дырявые».
- «Найдена уязвимость» ≠ «мы уязвимы». Нужно выяснить: используется ли компонент, попадает ли он в то, что запускается.
- База уязвимостей пополняется каждый день. Проект, который никто не трогал, со временем «краснеет» сам.
- **SBOM** — опись компонентов. Нужна, чтобы при новой громкой уязвимости за минуту ответить «есть ли она у нас».

Термин	Значение
CVE	идентификатор известной уязвимости, например `CVE-2019-19844`
Severity	тяжесть: LOW, MEDIUM, HIGH, CRITICAL
Fixed Version	версия, в которой уязвимость исправлена
Status `affected`	уязвимость есть, исправления пока нет
CycloneDX	формат SBOM

Режим Trivy	Что проверяет
`trivy fs`	файлы проекта: списки зависимостей
`trivy config`	настройки: Dockerfile
`trivy image`	собранный образ целиком, вместе с операционной системой

Ход работы

```
cd ~/devsecops-lab
source .venv/Scripts/activate      # Windows, Git Bash
# source .venv/bin/activate        # Linux и macOS
mkdir -p reports
export MSYS_NO_PATHCONV=1          # только Windows, Git Bash
```

Команды Trivy длинные, поэтому один раз заведите сокращение (действует до закрытия терминала):

```
trivy() { docker run --rm -v "$PWD:/src" -v trivy-cache:/root/.cache/trivy -v /v
ar/run/docker.sock:/var/run/docker.sock aquasec/trivy:0.76.2 "$@"; }
```

Теперь `trivy …` запускает сканер версии 0.76.2 в контейнере. Каталог проекта виден ему как `/src`.

Шаг 1. Просканировать зависимости

```
trivy --version
trivy fs --scanners vuln /src
```

Первый запуск скачивает базу уязвимостей (1–2 минуты).

Что увидите: сводную таблицу, затем подробности.

Файл	Уязвимостей в контрольном запуске
`legacy-component/requirements.txt`	44
`requirements.txt`	1

У вас чисел будет **больше** — база с тех пор пополнилась. Это нормально.

В отчёт: версия Trivy, дата, сводная таблица.

Шаг 2. Отфильтровать и прочитать

Только тяжёлые:

```
trivy fs --scanners vuln --severity HIGH,CRITICAL /src
```

Выберите по одной уязвимости для Django, requests, urllib3 и Jinja2 и заполните таблицу. Суть берите из колонки Title.

Библиотека	Версия	CVE	Тяжесть	Исправлено в	Суть одной фразой
requests	2.19.0	CVE-2018-18074	HIGH	2.20.0	при перенаправлении пароль уходит по незашифрованному каналу
Django					
urllib3					
Jinja2					

Сохраните полный отчёт в файл:

```
trivy fs --scanners vuln --format json --output /src/reports/trivy-fs-before.jso
n /src
```

В отчёт: таблица.

Шаг 3. Сформировать SBOM

```
trivy fs --format cyclonedx --output /src/reports/sbom-before.cdx.json /src
```

Откройте `reports/sbom-before.cdx.json` в редакторе, найдите `bomFormat` и список `components`.

Что увидите: CycloneDX, 9 компонентов: 7 библиотек и 2 файла со списками зависимостей.

В отчёт: формат и число компонентов.

Шаг 4. Разобраться, что из найденного опасно

44 уязвимости в `legacy-component` — это много. Проверьте три факта.

Факт 1. Использует ли код эти библиотеки?

```
grep -rnE "^(import|from) +(django|requests|urllib3)" backend/
```

Что увидите: пусто.

Факт 2. Попадают ли они в образ? Откройте `Dockerfile` — какой файл передаётся в `pip install`? Откройте `.dockerignore` — есть ли там `legacy-component`?

Факт 3. Есть ли они в собранном образе?

```
docker build -t devsecops-lab:local .
docker run --rm devsecops-lab:local pip list
```

Что увидите: Flask, Werkzeug, Jinja2 версии 3, psycopg, gunicorn. Django, requests и urllib3 в списке нет.

Запишите вывод по образцу:

> Компонент `legacy-component` есть в репозитории, но в работающее приложение не входит: (три доказательства). Для сервиса риск …, для проекта риск … . Решение: … .

Подсказка про риск для проекта: что будет, если кто-то этот файл установит? Как постоянный «красный» отчёт влияет на внимание к новым находкам?

В отчёт: три проверки и вывод.

Шаг 5. Включить порог

Пока сканер только показывал таблицы. Чтобы он мог остановить конвейер, ему нужен порог:

```
trivy fs --scanners vuln --severity HIGH,CRITICAL --exit-code 1 /src
echo "Код: $?"
```

`--exit-code 1` — вернуть ошибку, если найдены уязвимости указанной тяжести.

Что увидите: `Код: 1`. С таким результатом конвейер остановил бы изменение.

Без `--exit-code` Trivy всегда возвращает 0 — как Semgrep без `--error` в ЛР2.

Шаг 6. Применить исправление и пройти порог

Два разных решения для двух ситуаций.

Ненужный компонент — удалить:

```
git rm -r legacy-component
```

Нужную библиотеку — обновить, порт закрыть, права урезать. Готовые файлы:

```
cp -r ../secure-development-labs/solutions/lab4/. .
git diff
```

Что увидите в изменениях:

Файл	Изменение	Зачем
`requirements.txt`	Flask 3.0.3 → 3.1.3	версия с исправленной уязвимостью
`Dockerfile`	добавлены `useradd` и `USER labuser`	приложение работает не от root
`compose.yaml`	убран порт `5432` у базы данных	база недоступна снаружи

После обновления библиотеки обязательно прогоните тесты — новая версия могла что-то сломать:

```
python -m pip install -r requirements.txt
pytest -q
```

Что увидите: `7 passed`.

Повторите сканирование с порогом:

```
trivy fs --scanners vuln --severity HIGH,CRITICAL --exit-code 1 /src
echo "Код: $?"
```

Что увидите: `requirements.txt │ pip │ 0`, `Код: 0`.

Объясните в отчёте выбор порога: почему в конвейере блокируют HIGH и CRITICAL, а не все находки подряд?

Если в `requirements.txt` нашлась HIGH — значит, после выхода методички опубликовали новую уязвимость. Запишите CVE и Fixed Version в отчёт и сообщите преподавателю: так это и происходит в жизни.

В отчёт: оба вывода с кодами (до и после), объяснение порога.

Шаг 7. Проверить Dockerfile

Сначала — прежний Dockerfile (он ещё хранится в последнем коммите):

```
git show HEAD:Dockerfile > reports/Dockerfile
trivy config /src/reports/Dockerfile
```

Что увидите: DS-0002 (HIGH) — не задан `USER`, контейнер работает от root.

Теперь текущий:

```
trivy config --exit-code 1 /src/Dockerfile
echo "Код: $?"
```

Что увидите: `0` находок, `Код: 0`.

Убедитесь на деле:

```
docker run --rm devsecops-lab:local id
docker compose up -d --build --wait
docker compose exec web id
curl http://127.0.0.1:5000/health
```

Что увидите: старый образ — `uid=0(root)`; новый — `uid=1000(labuser)`; приложение отвечает `"status":"ok"` и работает с базой, хотя её порт наружу закрыт.

Обратите внимание: открытый порт базы данных Trivy **не показал** — файлы Compose он не проверяет. Эту проблему вы нашли глазами в ЛР1.

В отчёт: DS-0002, вывод `id` до и после.

Шаг 8. Просканировать образ

```
trivy image --severity HIGH,CRITICAL devsecops-lab-web
```

Имя `devsecops-lab-web` дал Compose (каталог + сервис). Если каталог назван иначе — посмотрите имя в `docker images`.

Что увидите: десятки находок в пакетах Debian (в контрольном запуске — 49 HIGH), хотя `requirements.txt` чист. Они пришли из базового образа `python:3.11-slim` — это целая операционная система.

Только те, для которых есть исправление:

```
trivy image --severity HIGH,CRITICAL --ignore-unfixed devsecops-lab-web
```

Что увидите: единицы (в контрольном запуске — 2).

Ответьте письменно:

1. Сколько HIGH/CRITICAL всего и сколько из них можно исправить обновлением?
2. Почему нельзя поставить в конвейер условие «образ без единой HIGH»?
3. Какой порог для образа разумен? (Подсказка: `--ignore-unfixed`.)

Остановите приложение:

```
docker compose down -v
```

В отчёт: числа и три ответа.

Шаг 9. SBOM «после» и правила приёмки

```
trivy fs --format cyclonedx --output /src/reports/sbom-after.cdx.json /src
```

Что увидите: 4 компонента вместо 9.

Заполните таблицу правил — что делать при каждом сигнале сканера. Она пригодится в ЛР5.

Сигнал	Блокировать изменение?	Действие
HIGH/CRITICAL в зависимостях, исправление есть	да	обновить, прогнать тесты
HIGH/CRITICAL в зависимостях, исправления нет		
Уязвимость в базовом образе без исправления		
Находка в Dockerfile		
Сканер не смог скачать базу уязвимостей		

Над последней строкой подумайте особенно: если сканер не отработал, проверка пройдена или не выполнена?

В отчёт: таблица.

Шаг 10. Как это встраивается в CI

```
- name: Scan dependencies
  run: docker run --rm -v "$PWD:/src" aquasec/trivy:0.76.2 fs --scanners vuln --
severity HIGH,CRITICAL --exit-code 1 /src
- name: Check Dockerfile
  run: docker run --rm -v "$PWD:/src" aquasec/trivy:0.76.2 config --exit-code 1
/src
- name: Generate SBOM
  run: docker run --rm -v "$PWD:/src" aquasec/trivy:0.76.2 fs --format cyclonedx
 --output /src/reports/sbom.cdx.json /src
```

Те же три команды. Версия сканера указана точно: сканер — такая же зависимость, как библиотека, и `latest` означает «неизвестно что».

Шаг 11. Сохранить результат

```
git add -A
git commit -m "lab4: update Flask, drop unused legacy manifest, non-root contain
er, close DB port"
git log --oneline
```

Что увидите: хук показал `Passed`, в истории пять коммитов.

Отчёт

1. Версия Trivy, дата, сводная таблица первого сканирования.
2. Таблица четырёх уязвимостей.
3. SBOM до и после: формат, число компонентов.
4. Разбор `legacy-component`: три проверки и вывод.
5. Повторное сканирование с порогом, объяснение порога.
6. DS-0002, вывод `id` до и после.
7. Сканирование образа, три ответа.
8. Таблица правил приёмки.

Контрольные вопросы

1. Чем SCA отличается от SAST?
2. Почему тяжесть HIGH не означает автоматически высокий риск для проекта?
3. Зачем нужен SBOM, если есть сканер?
4. Почему число находок в неизменном проекте растёт со временем?
5. Что делает параметр `--exit-code 1` и зачем он в конвейере?
6. Почему версию сканера указывают точно, а не `latest`?

Оценивание

Критерий	Баллы
Сканирование зависимостей, таблица уязвимостей	20
SBOM	10
Разбор применимости (`legacy-component`)	20
Порог и коды завершения	15
Dockerfile и образ	20
Правила приёмки, контрольные вопросы	15
Итого	**100**